

Notebook Native Agent in Action

```
In [1]: from notebook_native_agent import start_agent, stop_agent, agent_status, agent_symbols, pause_agent, resume_agent, get_state
start_agent()
```

⚡ Notebook agent active

provider: openai

auto-execute: on

fallback: on

LLM cell classifier: off

Notebook Native Agent

Write Python normally, or write a natural-language request in a cell. The agent shares the live notebook context, proposes code, and can safely auto-run allowed cases.

Try a natural-language cell like:

show me a funny fractal

copy

plot this: z = sin(x) * cos(y) (use plotly)

copy

I have this: tips.csv

copy

How it works

1 Stay in the notebook — write Python or natural-language requests directly in code cells; the router turns non-Python text into agent requests.

2 Work with live context — the agent tracks notebook symbols, reusable helpers, object summaries, and prior executions.

3 Preview or auto-run — generated code is shown, safety-checked, tracked, and auto-run only when allowed.

Fallback mode

Demo fallback is enabled. If no brain model/API key is available, the agent shows one fixed safe example: a graph of the sine function.

```
In [2]: show me a funny fractal
```

Notebook Native Agent

What I understood

Display a humorous fractal visualization in the notebook.

Proposed solution

I'll generate a colorful Mandelbrot-style fractal, then make it look funny by overlaying simple cartoon features like googly eyes and a smile. I create reusable symbols `funny_fractal_image`, `funny_fractal_extent`, and `funny_fractal_fig` in case you want to tweak or reuse the plot afterward.

Code preview

[copy](#)

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Arc
from IPython.display import display

def make_funny_fractal(width: int = 900, height: int = 600, max_iter: int = 140):
    """Create a colorful Mandelbrot-style fractal image and its plot extent."""
    x_min, x_max = -2.2, 1.2
    y_min, y_max = -1.3, 1.3

    x = np.linspace(x_min, x_max, width)
    y = np.linspace(y_min, y_max, height)
    c = x[None, :] + 1j * y[:, None]
    z = np.zeros_like(c)
    escape_iter = np.zeros(c.shape, dtype=int)
    active = np.ones(c.shape, dtype=bool)

    for i in range(max_iter):
        z[active] = z[active] * z[active] + c[active]
        escaped_now = np.abs(z) > 2
        just_escaped = escaped_now & active
        escape_iter[just_escaped] = i + 1
        active &= ~escaped_now
        if not active.any():
            break

    escape_iter[escape_iter == 0] = max_iter
    smooth = escape_iter.astype(float)
    mask = escape_iter < max_iter
    smooth[mask] = smooth[mask] + 1 - np.log2(np.log(np.abs(z[mask]) + 1e-12))

    return smooth, (x_min, x_max, y_min, y_max)

funny_fractal_image, funny_fractal_extent = make_funny_fractal()

funny_fractal_fig, ax = plt.subplots(figsize=(11, 7), facecolor="black")
ax.set_facecolor("black")
ax.imshow(
    funny_fractal_image,
    extent=funny_fractal_extent,
    origin="lower",
    cmap="turbo",
    interpolation="bilinear",
)

# Add googly eyes
left_eye = Circle((-1.15, 0.55), 0.18, facecolor="white", edgecolor="black", lw=2, alpha=0.95)
right_eye = Circle((-0.55, 0.55), 0.18, facecolor="white", edgecolor="black", lw=2, alpha=0.95)
left_pupil = Circle((-1.10, 0.49), 0.06, facecolor="black")
right_pupil = Circle((-0.60, 0.60), 0.06, facecolor="black")
ax.add_patch(left_eye)
ax.add_patch(right_eye)
ax.add_patch(left_pupil)
ax.add_patch(right_pupil)

# Add a smile
smile = Arc((-0.85, -0.10), 0.95, 0.65, theta1=200, theta2=340, color="white", lw=3)
ax.add_patch(smile)

# Tiny party hat for extra silliness
hat_x = [-0.98, -0.78, -0.58]
hat_y = [0.95, 1.28, 0.95]
ax.fill(hat_x, hat_y, color="#ff4d6d", alpha=0.9)
ax.plot([-0.98, -0.78, -0.58, -0.98], [0.95, 1.28, 0.95, 0.95], color="white", lw=1.5)
ax.scatter([-0.78], [1.31], s=60, color="#ffd166", zorder=5)

ax.set_title("A Funny Fractal", color="white", fontsize=18, pad=12)
```

```
ax.set_xticks([])
ax.set_yticks([])
for spine in ax.spines.values():
    spine.set_visible(False)

fig = funny_fractal_fig
display(fig)
plt.close(fig)
```

Execution

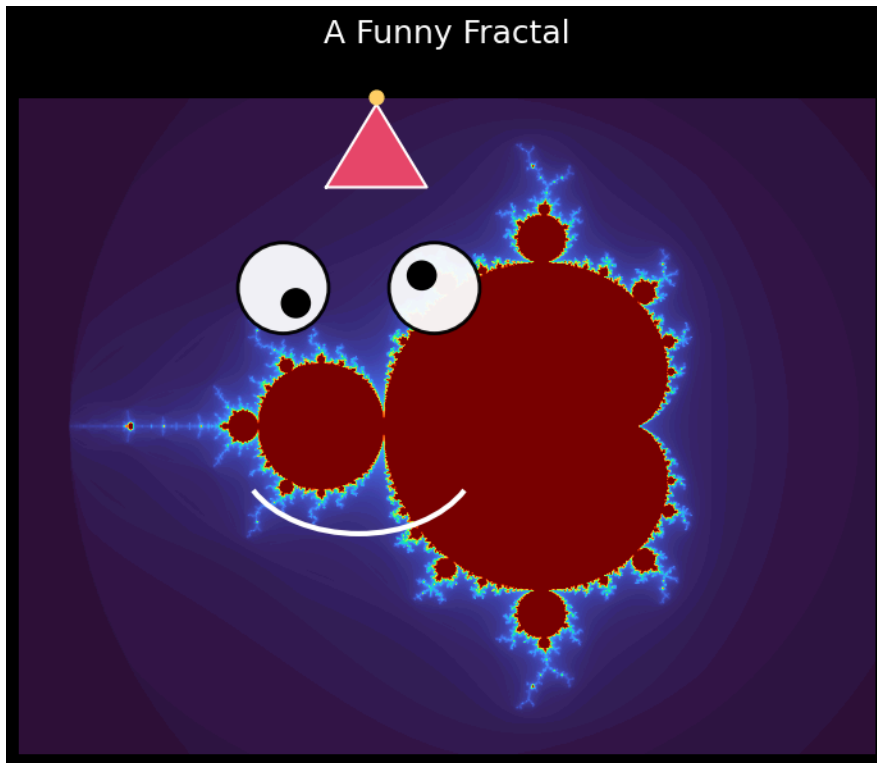
auto-execute via gpt-5.4

Selected context

No prior notebook context looked necessary.

Safety notes

- Looks like an in-memory inspection or plotting task.



In [11]: `stop_agent()`

Notebook Native Agent stopped

All agent hooks were detached. Natural-language routing and notebook tracking are now off. Use `start_agent()` for a fresh start.

In [12]: `start_agent()`

Notebook Native Agent

Write Python normally, or write a natural-language request in a cell. The agent shares the live notebook context, proposes code, and can safely auto-run allowed cases.

Try a natural-language cell like:

show me a funny fractal

copy

plot this: $z = \sin(x) * \cos(y)$ (use plotly)

copy

I have this: tips.csv

copy

How it works

- 1 Stay in the notebook** — write Python or natural-language requests directly in code cells; the router turns non-Python text into agent requests.
- 2 Work with live context** — the agent tracks notebook symbols, reusable helpers, object summaries, and prior executions.
- 3 Preview or auto-run** — generated code is shown, safety-checked, tracked, and auto-run only when allowed.

Fallback mode

Demo fallback is enabled. If no brain model/API key is available, the agent shows one fixed safe example: a graph of the sine function.

In [13]: `plot this:  $z = \sin(x) * \cos(y)$`

Notebook Native Agent

What I understood

Plot the function $z = \sin(x) * \cos(y)$ as a surface over x and y .

Proposed solution

Create a 3D surface plot on a reasonable grid for x and y , compute $z = \sin(x) * \cos(y)$, and render it explicitly. I reuse the existing notebook symbols `np`, `plt`, and `display` rather than introducing extra dependencies or relying on unknown helper signatures.

Code preview

```
x = np.linspace(-2 * np.pi, 2 * np.pi, 200)
y = np.linspace(-2 * np.pi, 2 * np.pi, 200)
X, Y = np.meshgrid(x, y)
Z = np.sin(X) * np.cos(Y)

fig = plt.figure(figsize=(9, 6))
ax = fig.add_subplot(111, projection="3d")
surface = ax.plot_surface(X, Y, Z, cmap="viridis", edgecolor="none")
ax.set_title("z = sin(x) * cos(y)")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")
fig.colorbar(surface, ax=ax, shrink=0.7, pad=0.1, label="z")

display(fig)
plt.close(fig)
```

Execution

auto-execute via gpt-5.4

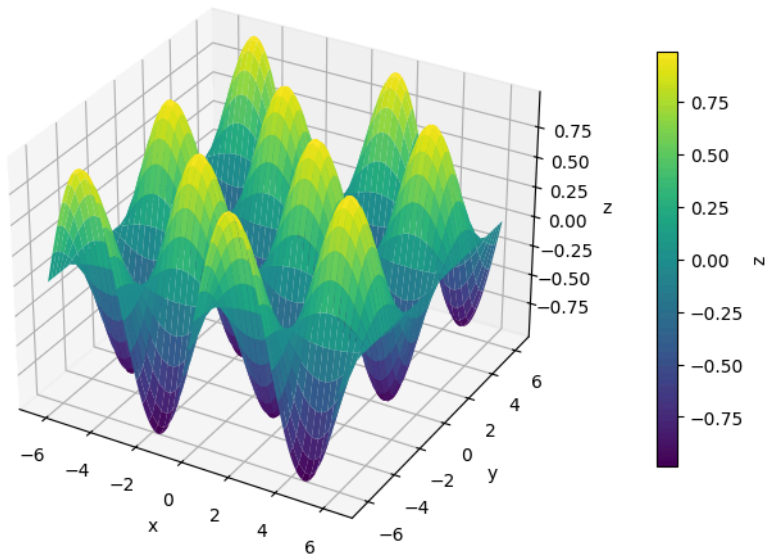
Selected context

No prior notebook context looked necessary.

Safety notes

- Looks like an in-memory inspection or plotting task.

$$z = \sin(x) * \cos(y)$$



In [14]: make a function **for** adding a plane $z =$ a given value **with** a given color

Notebook Native Agent

What I understood

Create a reusable function that adds a horizontal plane at a chosen z value with a chosen color.

Proposed solution

Define a small helper that adds a constant- z surface to an existing 3D Matplotlib axis. **Reuse decision:** it defaults to the notebook's existing X and Y meshgrid when available, and falls back to rebuilding a meshgrid from existing x and y arrays if needed. The function returns the created surface so it can be further customized later.

Code preview

copy

```
def add_constant_z_plane(ax, z_value: float, color: str, X_grid=None, Y_grid=None, alpha: float = 0.35, **plot_surface_kwargs):
    """Add a horizontal plane  $z = z\_value$  to an existing 3D Matplotlib axis."""
    if X_grid is None or Y_grid is None:
        if "X" in globals() and "Y" in globals():
            X_grid, Y_grid = X, Y
        elif "x" in globals() and "y" in globals():
            X_grid, Y_grid = np.meshgrid(x, y)
        else:
            raise ValueError("Provide X_grid and Y_grid, or define X/Y or x/y in the notebook.")

    Z_plane = np.full_like(X_grid, fill_value=float(z_value), dtype=float)
    plane = ax.plot_surface(
        X_grid,
        Y_grid,
        Z_plane,
        color=color,
        alpha=alpha,
        shade=False,
        **plot_surface_kwargs,
    )
    return plane

print("Created function: add_constant_z_plane")
```

Execution

auto-execute via gpt-5.4

Selected context

- The notebook already has a 3D plot setup: `ax` is an `Axes3D` with title `z = sin(x) * cos(y)` and `x/y/z` labels; `fig` is a `Figure` with 2 axes.
- Arrays `x`, `y`, `X`, `Y`, and `Z` are present; `X`, `Y`, and `Z` come from the existing surface plot for `z = sin(x) * cos(y)`.
- A surface object already exists: `surface` is a `Poly3DCollection`.
- No function for adding a plane at a constant `z` value or specifying a plane color is present in the selected context.

Safety notes

- No dangerous patterns detected by the basic rules-first safety gate.

Created function: add_constant_z_plane

In [15]: add the plane $z=0$ color red

Notebook Native Agent

What I understood

Add a red horizontal plane at $z=0$ to the current 3D plot.

Proposed solution

Reuse the existing `add_constant_z_plane(...)` helper and the current `ax`, `X`, and `Y` grid symbols so the plane matches the plotted surface domain without rebuilding the figure.

Code preview

```
plane = add_constant_z_plane(ax, z_value=0.0, color="red", X_grid=X, Y_grid=Y)
display(ax.figure)
```

Execution

auto-execute via gpt-5.4

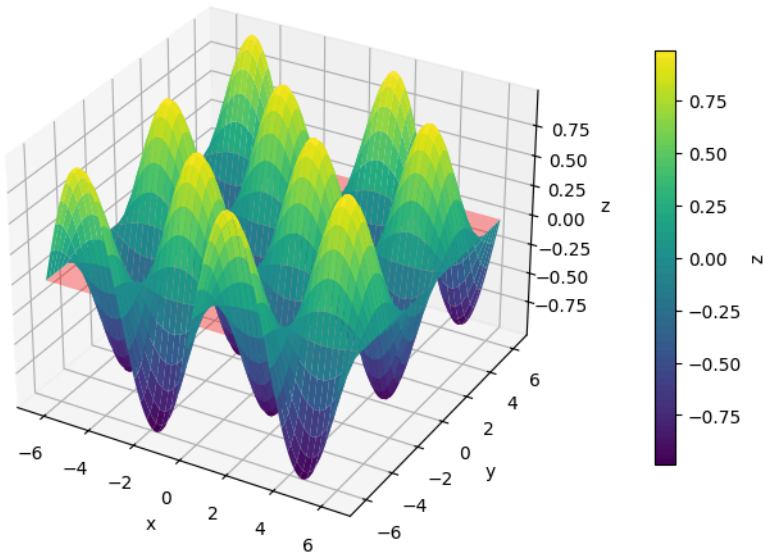
Selected context

- `add_constant_z_plane(ax, z_value, color, X_grid=None, Y_grid=None, alpha=0.35, **plot_surface_kwargs)` exists; its docstring says it adds a horizontal plane $z = z_value$ to an existing 3D Matplotlib axis.
- `ax` is an `Axes3D` with title $z = \sin(x) * \cos(y)$ and x/y/z labels set; a `surface` (`Poly3DCollection`) is already present.
- `x`, `y`, `X`, `Y`, and `Z` arrays are already in the notebook context; `Z` is an ndarray with values consistent with the plotted surface.
- The selected context does not show the implementation body of `add_constant_z_plane` or any explicit plane-at- $z=0$ call.

Safety notes

- Looks like an in-memory inspection or plotting task.

$$z = \sin(x) * \cos(y)$$



In [16]: `pause_agent()`

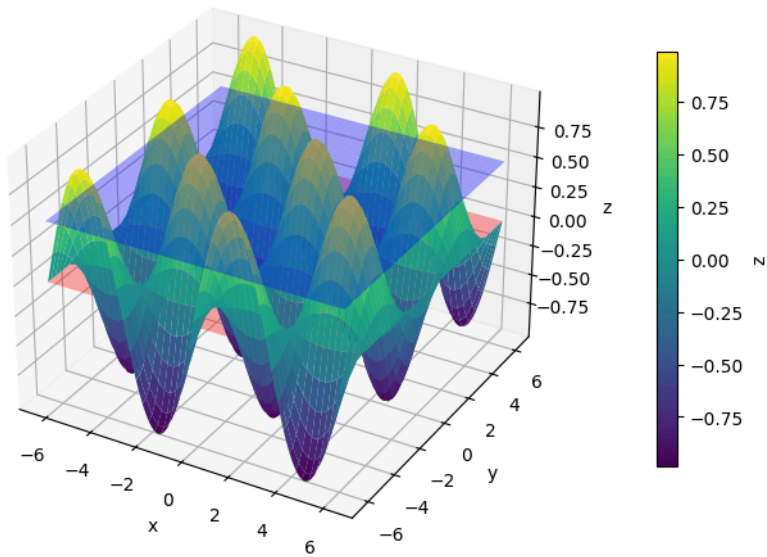
Notebook Native Agent paused

Natural-language routing is paused. Python cells still run normally and are still tracked. You can now debug manually, edit `notebook_native_agent.config.json` or `openai.config.json`, then call `resume_agent()` to continue.

With the agent paused, let's manually call the generated function to add a new blue plan.

In [17]: `plane = add_constant_z_plane(ax, z_value=0.5, color="blue", X_grid=X, Y_grid=Y)`
`display(ax.figure)`

$$z = \sin(x) * \cos(y)$$



```
In [18]: resume_agent()
```

Notebook Native Agent resumed

Natural-language routing is active again. Configuration was reloaded, and existing notebook memory was preserved.

```
In [19]: how many planes are in the plot?
```

Notebook Native Agent

What I understood

Determine how many plane surfaces are in the current 3D plot.

Proposed solution

Based on the notebook history, the most reasonable count is **2 planes**:

- a **blue** plane at `z = 0.5`
- a **red** plane at `z = 0.0`

Key reuse decision: I relied on the existing executed cells rather than inventing new plot state. The `plane` symbol only refers to the most recently added `Poly3DCollection`, so it does **not** imply there is only one plane overall. If both were added to the same `ax` without clearing it, the plot contains **2** planes.

Code preview

```
# No code generated because the user requested analysis/explanation only.
```

Execution

preview only

Selected context

- `plane` is a `Poly3DCollection` object in the context; the selected context does not explicitly state how many total planes are currently rendered.
- One executed cell added a plane with `add_constant_z_plane(ax, z_value=0.5, color="blue", X_grid=X, Y_grid=Y)`.
- Another executed cell added a plane with `add_constant_z_plane(ax, z_value=0.0, color="red", X_grid=X, Y_grid=Y)`.
- The selected context does not include any plot metadata confirming whether both additions are simultaneously present in the displayed figure.

Safety notes

- No dangerous patterns detected by the basic rules-first safety gate.

Manual step

This code was stored as proposal #1. If you copy it into a normal Python cell and run it, the historian will mark it as `agent_proposed_then_user_executed`.

```
In [7]: stop_agent()
```

Notebook Native Agent stopped

All agent hooks were detached. Natural-language routing and notebook tracking are now off. Use `start_agent()` for a fresh start.

In [2]: start_agent()

⚡ Notebook agent active

provider: openai

auto-execute: on

fallback: on

LLM cell classifier: off

Notebook Native Agent

Write Python normally, or write a natural-language request in a cell. The agent shares the live notebook context, proposes code, and can safely auto-run allowed cases.

Try a natural-language cell like:

show me a funny fractal

copy

plot this: `z = sin(x) * cos(y)` (use plotly)

copy

I have this: tips.csv

copy

How it works

- 1 Stay in the notebook** — write Python or natural-language requests directly in code cells; the router turns non-Python text into agent requests.
- 2 Work with live context** — the agent tracks notebook symbols, reusable helpers, object summaries, and prior executions.
- 3 Preview or auto-run** — generated code is shown, safety-checked, tracked, and auto-run only when allowed.

Fallback mode

Demo fallback is enabled. If no brain model/API key is available, the agent shows one fixed safe example: a graph of the sine function.

In [3]: I have this: tips.csv

Notebook Native Agent

What I understood

Load the `tips.csv` file into the notebook and give a quick reusable overview.

Proposed solution

I'll load `tips.csv` into a dataframe, then create a few reusable summary objects for follow-up work:

- `tips_df`: the loaded dataset
- `tips_columns`: column names
- `tips_dtypes`: dtype summary
- `tips_missing_summary`: missing values per column
- `numeric_cols` / `categorical_cols`: useful column groups

The code also displays a preview and the summaries so you can immediately inspect what was loaded. If the file is not found, it raises a clear error.

Code preview

copy

```
from pathlib import Path

import pandas as pd
from IPython.display import display

def load_csv_with_basic_summary(csv_path: str) -> tuple[pd.DataFrame, list[str], pd.Series, pd.Series, list[str], list[str]]:
    """Load a CSV and prepare a few reusable summary objects."""
    path = Path(csv_path)
    if not path.exists():
        raise FileNotFoundError(f"Could not find CSV file: {path.resolve()}")

    df = pd.read_csv(path)
    columns = df.columns.tolist()
    dtypes = df.dtypes.astype(str)
    missing_summary = df.isna().sum().sort_values(ascending=False)
    numeric_cols = df.select_dtypes(include="number").columns.tolist()
    categorical_cols = df.select_dtypes(exclude="number").columns.tolist()
    return df, columns, dtypes, missing_summary, numeric_cols, categorical_cols

tips_df, tips_columns, tips_dtypes, tips_missing_summary, numeric_cols, categorical_cols = load_csv_with_basic_summary("tips.csv")

print(f"Loaded tips.csv with shape: {tips_df.shape}")
display(tips_df.head())
display(pd.DataFrame({"dtype": tips_dtypes, "missing": tips_missing_summary.reindex(tips_columns).fillna(0).astype(int)}))
print("numeric_cols:", numeric_cols)
print("categorical_cols:", categorical_cols)
```

Execution

auto-execute via gpt-5.4

Selected context

No prior notebook context looked necessary.

Safety notes

- Looks like an in-memory inspection or plotting task.

Loaded tips.csv with shape: (244, 7)

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

	dtype	missing
total_bill	float64	0
tip	float64	0
sex	object	0
smoker	object	0
day	object	0
time	object	0
size	int64	0

```
numeric_cols: ['total_bill', 'tip', 'size']
categorical_cols: ['sex', 'smoker', 'day', 'time']
```

```
In [4]: make a quick EDA on this tips
```

Notebook Native Agent

What I understood

Do a quick exploratory data analysis of the tips dataset already in the notebook.

Proposed solution

Reuse the existing `tips_df` and summary symbols when available, then build a lightweight EDA pass that is easy to rerun.

Approach

- Reuse `tips_df`, `tips_dtypes`, `tips_missing_summary`, `numeric_cols`, and `categorical_cols` if they already exist.
- Fall back to loading `tips.csv` with `load_csv_with_basic_summary(...)` only if needed.
- Create a reusable `tips_analysis_df` copy and add `tip_pct` ($\text{tip} / \text{total_bill} * 100$) for a more informative tip analysis.
- Display:
 - dataset overview and preview
 - dtypes and missing values
 - numeric summary stats
 - correlation matrix
 - categorical value counts
 - grouped summary by `day` and `time`
- Render a compact 2x2 figure for key distributions and relationships.

Reusable symbols created

- `tips_analysis_df`
- `tips_shape_summary`
- `tips_numeric_summary_df`
- `tips_correlation_df`
- `tips_group_summary_df`
- `tips_categorical_counts`

Code preview

[copy](#)

```
import matplotlib.pyplot as plt

if "tips_df" not in globals():
    csv_path = Path("tips.csv")
    if not csv_path.exists():
        raise FileNotFoundError("tips.csv was not found in the current working directory.")

    if "load_csv_with_basic_summary" in globals():
        tips_df, tips_columns, tips_dtypes, tips_missing_summary, numeric_cols, categorical_cols = load_csv_with_basic_summary(str(csv_path))
    else:
        tips_df = pd.read_csv(csv_path)
        tips_columns = tips_df.columns.tolist()
        tips_dtypes = tips_df.dtypes.astype(str)
        tips_missing_summary = tips_df.isna().sum()
        numeric_cols = tips_df.select_dtypes(include="number").columns.tolist()
        categorical_cols = tips_df.select_dtypes(exclude="number").columns.tolist()

# Refresh reusable summaries so they stay in sync with the current dataframe.
tips_columns = tips_df.columns.tolist()
tips_dtypes = tips_df.dtypes.astype(str)
tips_missing_summary = tips_df.isna().sum()
numeric_cols = tips_df.select_dtypes(include="number").columns.tolist()
categorical_cols = tips_df.select_dtypes(exclude="number").columns.tolist()

tips_analysis_df = tips_df.copy()
if {"tip", "total_bill"}.issubset(tips_analysis_df.columns):
    tips_analysis_df["tip_pct"] = (tips_analysis_df["tip"] / tips_analysis_df["total_bill"] * 100).round(2)

tips_shape_summary = pd.Series(
    {
        "rows": len(tips_analysis_df),
        "columns": tips_analysis_df.shape[1],
        "duplicate_rows": int(tips_analysis_df.duplicated().sum()),
    },
    name="value",
)

tips_numeric_summary_df = tips_analysis_df.select_dtypes(include="number").describe().T.round(2)
tips_correlation_df = tips_analysis_df.select_dtypes(include="number").corr().round(2)

tips_categorical_counts = {
    col: tips_analysis_df[col].value_counts(dropna=False).rename_axis(col).reset_index(name="count")
    for col in categorical_cols
}

if {"day", "time", "total_bill", "tip", "size"}.issubset(tips_analysis_df.columns):
    agg_map = {
```

```

    "party_count": ("size", "count"),
    "avg_total_bill": ("total_bill", "mean"),
    "avg_tip": ("tip", "mean"),
    "avg_party_size": ("size", "mean"),
}

if "tip_pct" in tips_analysis_df.columns:
    agg_map["avg_tip_pct"] = ("tip_pct", "mean")

tips_group_summary_df = (
    tips_analysis_df.groupby(["day", "time"], dropna=False)
    .agg(**agg_map)
    .round(2)
    .reset_index()
)
else:
    tips_group_summary_df = pd.DataFrame()

print("Quick EDA for tips_df")
display(tips_shape_summary.to_frame())
display(tips_analysis_df.head())
display(tips_dtypes.rename("dtype").to_frame())
display(tips_missing_summary.rename("missing_count").to_frame())
display(tips_numeric_summary_df)

if not tips_correlation_df.empty:
    display(tips_correlation_df)

for col, counts_df in tips_categorical_counts.items():
    print(f"Value counts for {col}")
    display(counts_df)

if not tips_group_summary_df.empty:
    print("Grouped summary by day and time")
    display(tips_group_summary_df)

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.ravel()

if "total_bill" in tips_analysis_df.columns:
    tips_analysis_df["total_bill"].plot(kind="hist", bins=20, ax=axes[0], title="Total Bill Distribution")
    axes[0].set_xlabel("total_bill")
else:
    axes[0].set_visible(False)

if "tip" in tips_analysis_df.columns:
    tips_analysis_df["tip"].plot(kind="hist", bins=20, ax=axes[1], title="Tip Distribution", color="tab:orange")
    axes[1].set_xlabel("tip")
else:
    axes[1].set_visible(False)

if {"total_bill", "tip"}.issubset(tips_analysis_df.columns):
    axes[2].scatter(tips_analysis_df["total_bill"], tips_analysis_df["tip"], alpha=0.7)
    axes[2].set_title("Tip vs Total Bill")
    axes[2].set_xlabel("total_bill")
    axes[2].set_ylabel("tip")
else:
    axes[2].set_visible(False)

if {"day", "tip_pct"}.issubset(tips_analysis_df.columns):
    preferred_order = ["Thur", "Fri", "Sat", "Sun"]
    present_days = tips_analysis_df["day"].dropna().astype(str).unique().tolist()
    day_order = [day for day in preferred_order if day in present_days]
    if not day_order:
        day_order = sorted(present_days)
    avg_tip_pct_by_day = (
        tips_analysis_df.groupby("day", dropna=False)["tip_pct"].mean().reindex(day_order).dropna().round(2)
    )
    avg_tip_pct_by_day.plot(kind="bar", ax=axes[3], title="Average Tip % by Day", color="tab:green")
    axes[3].set_xlabel("day")
    axes[3].set_ylabel("avg tip %")
else:
    axes[3].set_visible(False)

fig.tight_layout()
display(fig)
plt.close(fig)

print("Created reusable symbols: tips_analysis_df, tips_shape_summary, tips_numeric_summary_df, tips_correlation_df, tips_group_summary_df, tips_categorical_counts")

```

Execution

auto-execute via gpt-5.4

Selected context

- `tips.csv` is the target file from the prior generated code context.
- Available imports/symbols include `Path`, `pandas` as `pd`, and `display`.
- A helper function `load_csv_with_basic_summary(csv_path: str)` exists; it returns a tuple: `DataFrame`, `list[str]`, `Series`, `Series`, `list[str]`, `list[str]`.
- The helper's docstring says it "Load[s] a CSV and prepare[s] a few reusable summary objects."

Safety notes

- Looks like an in-memory inspection or plotting task.

Quick EDA for `tips_df`

value	
rows	244
columns	8
duplicate_rows	1

	total_bill	tip	sex	smoker	day	time	size	tip_pct
0	16.99	1.01	Female	No	Sun	Dinner	2	5.94
1	10.34	1.66	Male	No	Sun	Dinner	3	16.05
2	21.01	3.50	Male	No	Sun	Dinner	3	16.66
3	23.68	3.31	Male	No	Sun	Dinner	2	13.98
4	24.59	3.61	Female	No	Sun	Dinner	4	14.68

dtype	
total_bill	float64
tip	float64
sex	object
smoker	object
day	object
time	object
size	int64

missing_count	
total_bill	0
tip	0
sex	0
smoker	0
day	0
time	0
size	0

	count	mean	std	min	25%	50%	75%	max
total_bill	244.0	19.79	8.90	3.07	13.35	17.80	24.13	50.81
tip	244.0	3.00	1.38	1.00	2.00	2.90	3.56	10.00
size	244.0	2.57	0.95	1.00	2.00	2.00	3.00	6.00
tip_pct	244.0	16.08	6.11	3.56	12.91	15.48	19.15	71.03

	total_bill	tip	size	tip_pct
total_bill	1.00	0.68	0.60	-0.34
tip	0.68	1.00	0.49	0.34
size	0.60	0.49	1.00	-0.14
tip_pct	-0.34	0.34	-0.14	1.00

Value counts for `sex`

	sex	count
0	Male	157
1	Female	87

Value counts for `smoker`

smoker count		
0	No	151
1	Yes	93

Value counts for day

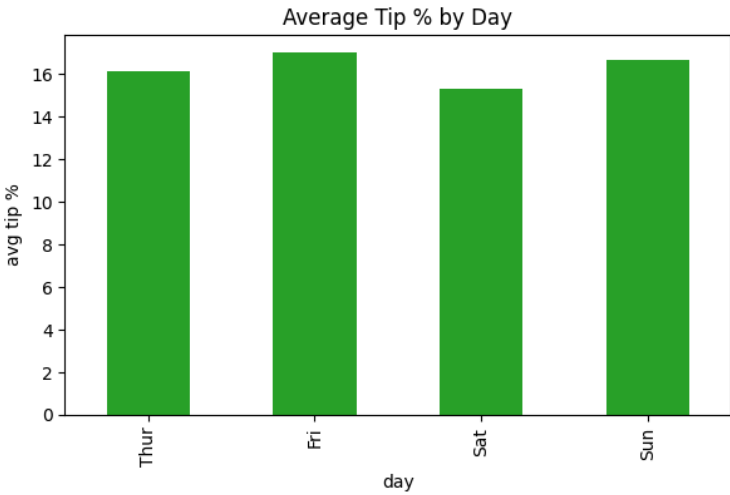
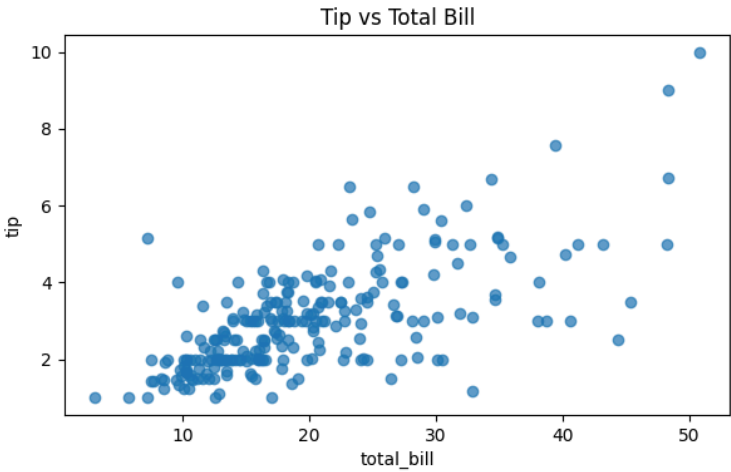
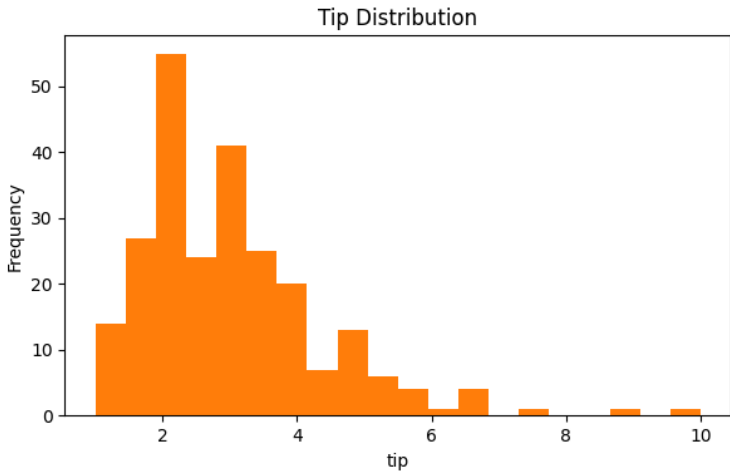
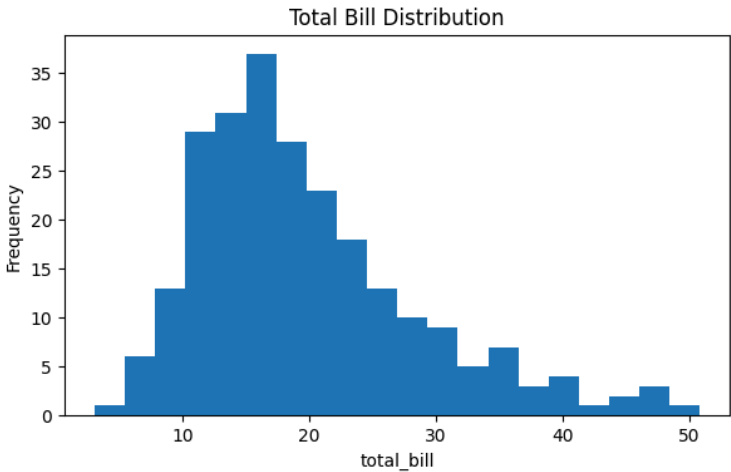
day count		
0	Sat	87
1	Sun	76
2	Thur	62
3	Fri	19

Value counts for time

time count		
0	Dinner	176
1	Lunch	68

Grouped summary by day and time

	day	time	party_count	avg_total_bill	avg_tip	avg_party_size	avg_tip_pct
0	Fri	Dinner	12	19.66	2.94	2.17	15.89
1	Fri	Lunch	7	12.85	2.38	2.00	18.88
2	Sat	Dinner	87	20.44	2.99	2.52	15.31
3	Sun	Dinner	76	21.41	3.26	2.84	16.69
4	Thur	Dinner	1	18.78	3.00	2.00	15.97
5	Thur	Lunch	61	17.66	2.77	2.46	16.13



Created reusable symbols: tips_analysis_df, tips_shape_summary, tips_numeric_summary_df, tips_correlation_df, tips_group_summary_df, tips_categorical_counts

```
In [3]: stop_agent()
```

Notebook Native Agent stopped

All agent hooks were detached. Natural-language routing and notebook tracking are now off. Use start_agent() for a fresh start.

```
In [4]: start_agent()
```

⚡ Notebook agent active

provider: openai

auto-execute: on

fallback: on

LLM cell classifier: off

Notebook Native Agent

Write Python normally, or write a natural-language request in a cell. The agent shares the live notebook context, proposes code, and can safely auto-run allowed cases.

Try a natural-language cell like:

show me a funny fractal

copy

plot this: $z = \sin(x) * \cos(y)$ (use plotly)

copy

I have this: tips.csv

copy

How it works

- 1 **Stay in the notebook** — write Python or natural-language requests directly in code cells; the router turns non-Python text into agent requests.
- 2 **Work with live context** — the agent tracks notebook symbols, reusable helpers, object summaries, and prior executions.
- 3 **Preview or auto-run** — generated code is shown, safety-checked, tracked, and auto-run only when allowed.

Fallback mode

Demo fallback is enabled. If no brain model/API key is available, the agent shows one fixed safe example: a graph of the sine function.

In [2]: Fa un graf cu orașe și rute, si apoi afiseaza cele mai scurte rute dintr-un oraș

Notebook Native Agent

What I understood

Construiesc un graf cu orașe și rute și afișez cele mai scurte drumuri dintr-un oraș ales.

Proposed solution

Voi crea un exemplu mic de rețea de orașe și rute ponderate, îl voi desena și apoi voi calcula drumurile minime dintr-un oraș sursă.

- **Presupunere explicită:** nu ai furnizat date, deci folosesc un set de rute exemplu pe care îl poți modifica ușor.
- **Reutilizare pentru notebook:** creez simboluri utile pentru pașii următori: `routes_df`, `city_graph`, `source_city`, `shortest_routes_df`.
- **Pași:**
 1. definesc rutele dintre orașe cu distanțe;
 2. construiesc un graf ponderat `networkx`;
 3. afișez graful cu etichete și costuri pe muchii;
 4. calculez cele mai scurte rute din `source_city` către toate celelalte orașe;
 5. afișez rezultatele într-un tabel ordonat după distanță.

Dacă vrei, ulterior putem înlocui rețeaua exemplu cu orașele și rutele tale reale.

Code preview

[copy](#)

```
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

try:
    import networkx as nx
except ImportError as exc:
    raise ImportError("Pachetul 'networkx' nu este instalat. Instalează-l cu: pip install networkx") from exc

routes_data = [
    ("București", "Ploiești", 60),
    ("București", "Pitești", 110),
    ("București", "Constanța", 225),
    ("Ploiești", "Brașov", 60),
    ("Ploiești", "Buzău", 70),
    ("Brașov", "Sibiu", 145),
    ("Brașov", "Târgu Mureș", 170),
    ("Pitești", "Sibiu", 120),
    ("Sibiu", "Cluj-Napoca", 170),
    ("Târgu Mureș", "Cluj-Napoca", 100),
    ("Buzău", "Iași", 260),
    ("Constanța", "Galați", 190),
    ("Galați", "Iași", 225),
]

routes_df = pd.DataFrame(routes_data, columns=["source", "target", "distance_km"])

def build_city_graph(routes_df: pd.DataFrame) -> nx.Graph:
    """Construiește un graf neorientat, ponderat, dintr-un tabel de rute."""
    graph = nx.Graph()
    for row in routes_df.iteruples(index=False):
        graph.add_edge(row.source, row.target, weight=row.distance_km)
    return graph

def shortest_routes_from_city(graph: nx.Graph, source: str) -> pd.DataFrame:
    """Returnează cele mai scurte rute și distanțe din orașul sursă către toate celelalte orașe."""
    lengths = nx.single_source_dijkstra_path_length(graph, source, weight="weight")
    paths = nx.single_source_dijkstra_path(graph, source, weight="weight")

    shortest_routes_df = pd.DataFrame(
        {
            "destination": list(lengths.keys()),
            "distance_km": list(lengths.values()),
            "path": [" -> ".join(paths[city]) for city in lengths.keys()],
        }
    )
    shortest_routes_df = shortest_routes_df.sort_values(["distance_km", "destination"]).reset_index(drop=True)
    return shortest_routes_df

city_graph = build_city_graph(routes_df)
source_city = "București"

if source_city not in city_graph:
    raise ValueError(f'Orașul sursă '{source_city}' nu există în graf.")
```



```

pos = nx.spring_layout(city_graph, seed=42, k=1.2)
edge_labels = nx.get_edge_attributes(city_graph, "weight")

fig, ax = plt.subplots(figsize=(11, 7))
nx.draw_networkx(
    city_graph,
    pos=pos,
    ax=ax,
    with_labels=True,
    node_color="#9ecae1",
    node_size=2200,
    font_size=10,
    edge_color="#6b6b6b",
    width=2,
)
nx.draw_networkx_edge_labels(
    city_graph,
    pos=pos,
    edge_labels=edge_labels,
    font_size=9,
    ax=ax,
)
ax.set_title("Graf de orașe și rute")
ax.axis("off")
display(fig)
plt.close(fig)

shortest_routes_df = shortest_routes_from_city(city_graph, source_city)
print(f"Cele mai scurte rute din {source_city}:")
display(shortest_routes_df)
print("Simboluri create: routes_df, city_graph, source_city, shortest_routes_df")

```

Execution

preview only

Selected context

No prior notebook context looked necessary.

Safety notes

- Manual review required because code contains: pip install

Manual step

This code was stored as proposal #1. If you copy it into a normal Python cell and run it, the historian will mark it as agent_proposed_then_user_executed.

```

In [4]: import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

try:
    import networkx as nx
except ImportError as exc:
    raise ImportError("Pachetul 'networkx' nu este instalat. Instalează-l cu: pip install networkx") from exc

routes_data = [
    ("București", "Ploiești", 60),
    ("București", "Pitești", 110),
    ("București", "Constanța", 225),
    ("Ploiești", "Brașov", 110),
    ("Ploiești", "Buzău", 70),
    ("Brașov", "Sibiu", 145),
    ("Brașov", "Târgu Mureș", 170),
    ("Pitești", "Sibiu", 120),
    ("Sibiu", "Cluj-Napoca", 170),
    ("Târgu Mureș", "Cluj-Napoca", 100),
    ("Buzău", "Iași", 260),
    ("Constanța", "Galați", 190),
    ("Galați", "Iași", 225),
]

routes_df = pd.DataFrame(routes_data, columns=["source", "target", "distance_km"])

def build_city_graph(routes_df: pd.DataFrame) -> nx.Graph:
    """Construiește un graf neorientat, ponderat, dintr-un tabel de rute."""
    graph = nx.Graph()
    for row in routes_df.itertuples(index=False):
        graph.add_edge(row.source, row.target, weight=row.distance_km)
    return graph

```

```

def shortest_routes_from_city(graph: nx.Graph, source: str) -> pd.DataFrame:
    """Returnează cele mai scurte rute și distanțe din orașul sursă către toate celelalte orașe."""
    lengths = nx.single_source_dijkstra_path_length(graph, source, weight="weight")
    paths = nx.single_source_dijkstra_path(graph, source, weight="weight")

    shortest_routes_df = pd.DataFrame(
        {
            "destination": list(lengths.keys()),
            "distance_km": list(lengths.values()),
            "path": [" -> ".join(paths[city]) for city in lengths.keys()],
        }
    )
    shortest_routes_df = shortest_routes_df.sort_values(["distance_km", "destination"]).reset_index(drop=True)
    return shortest_routes_df

city_graph = build_city_graph(routes_df)
source_city = "București"

if source_city not in city_graph:
    raise ValueError(f'Orașul sursă '{source_city}' nu există în graf.')

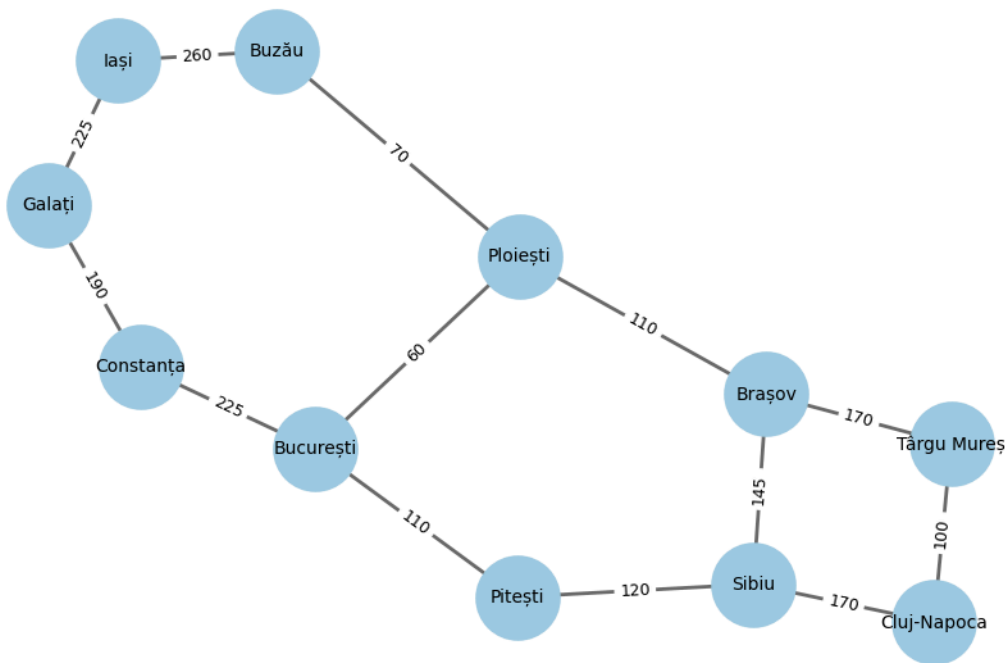
pos = nx.spring_layout(city_graph, seed=42, k=1.2)
edge_labels = nx.get_edge_attributes(city_graph, "weight")

fig, ax = plt.subplots(figsize=(11, 7))
nx.draw_networkx(
    city_graph,
    pos=pos,
    ax=ax,
    with_labels=True,
    node_color="#9ecae1",
    node_size=2200,
    font_size=10,
    edge_color="#6b6b6b",
    width=2,
)
nx.draw_networkx_edge_labels(
    city_graph,
    pos=pos,
    edge_labels=edge_labels,
    font_size=9,
    ax=ax,
)
ax.set_title("Graf de orașe și rute")
ax.axis("off")
display(fig)
plt.close(fig)

shortest_routes_df = shortest_routes_from_city(city_graph, source_city)
print(f"Cele mai scurte rute din {source_city}:")
display(shortest_routes_df)
print("Simboluri create: routes_df, city_graph, source_city, shortest_routes_df")

```

Graf de orașe și rute



Cele mai scurte rute din București:

	destination	distance_km	path
0	București	0	București
1	Ploiești	60	București -> Ploiești
2	Pitești	110	București -> Pitești
3	Buzău	130	București -> Ploiești -> Buzău
4	Brașov	170	București -> Ploiești -> Brașov
5	Constanța	225	București -> Constanța
6	Sibiu	230	București -> Pitești -> Sibiu
7	Târgu Mureș	340	București -> Ploiești -> Brașov -> Târgu Mureș
8	Iași	390	București -> Ploiești -> Buzău -> Iași
9	Cluj-Napoca	400	București -> Pitești -> Sibiu -> Cluj-Napoca
10	Galați	415	București -> Constanța -> Galați

Simboluri create: routes_df, city_graph, source_city, shortest_routes_df

```
In [ ]:
```